

Bi-connected components

Algorithm to find Dfn's and Low values

Algorithm Art(u,v)

//u is a start vertex for depth first search,v is its parent if any in the depth first spanning tree. If ssumed that global array dfn is initialized to zero and that the global variable num is intialized to 1, n is the no. of vertices in G

{

dfn[u]=num;

Low[u]=num;

num=num+1;

For each vertex w adjacent from u do

{

if dfn[w]=0) then

{

```
Art(w,u);  
Low[u]=min(low[u],Low[w])  
}  
else if(w!=u) then Low[u]=min(Low[u],dfn[w]);  
}  
}
```

Note: Using the L values the articulation points are computed when the condition is satisfied i.e, $L[w] \geq dfn[u]$

Algorithm to find biconnected components

Algorithm: Bicomp(u,v)

// u is a start vertex for depth first search. V is its parent if any in the depth first spanning tree. It is assumed that the global array num is initialized to 1, n is no. of vertices in G

{

dfn[u]=num;

L[u]=num;

num=num+1;

```
for each vertex w adjacent from u do:
{
if ((v!=w) and (dfn[w]<=dfn[u])) then
add(u,w) to the top of a stack s;
if(dfn[w]=0) then
{
if(L[w]>= dfn[u]) then
{
write("New bicomponent");
repeat
{
Delete an edge from top of the stack s;
let this edge be (x,y);
write(x,y);
```

```
}until(((x,y)=(u,w)) or ((x,y)=(w,u)));  
}  
Bicomp(w,u);  
L[u]:=min(L[u],L[w]);  
}  
else if(w!=v) then  
L[u]:=min(L[u],dfn[w]);  
}  
}
```